Резюме за стъпка 1 — Основи на обучението с подкрепление

Изследване на възможностите за приложение на изкуствения интелект в компютърни игри

Александър Андреев

Април 2026

Съдържание

1 Стъпка 1 — Основи на обучението с подкрепление (Reinforcement Learning) 1
1.1 Как обучението с подкрепление стана дисциплина 2
1.2 Основни принципи — Агент, Действие, Среда (Agent, Action, Environment) 3
1.3 Формулировката на MDP 4
1.4 Информационни множества — Когато не виждате всичко 5
1.5 Динамично програмиране — Когато имате модела 6
1.6 Обучение по времеви разлики — Учене без модел 7
1.7 DQN — Deep Q-Networks 8
1.8 PPO — Proximal Policy Optimization 10
1.9 Практическо валидиране — Собствени имплементации срещу Stable-Baselines3 12

1 Стъпка 1 — Основи на обучението с подкрепление (Reinforcement Learning)

Това е съкратено резюме на основните материали по обучение с подкрепление (reinforcement learning), разгледани в Стъпка 1. То служи за две цели: като бърз справочник по време на преминаването към следващите стъпки и като основен източник за синтезиране на финалния публичен доклад в Стъпка 15.

1.1 Как обучението с подкрепление стана дисциплина

Обучението с подкрепление не се появи изцяло завършено. Корените му се простират в няколко области, които до голяма степен не са знаели една за друга в продължение на десетилетия.

Най-ранната нишка идва от **животинската психология** (animal psychology) в началото на 1900-те — „Законът за ефекта“ на Торндайк (1911) предполага, че действията, последвани от удовлетворяващи резултати, стават по-вероятни. Това по същество е идеята за сигнала за възнаграждение (reward signal). Паралелна нишка минава през **теорията на оптималното управление** (optimal control theory) през 50-те години на миналия век, когато Белман разработва динамичното програмиране за решаване на последователни задачи за вземане на решения. Той въвежда функцията на стойността (value function) и принципа на оптималността, които стоят в основата на съвременния RL. Трета нишка идва от **ученето чрез проби и грешки** (trial-and-error learning) в ранните изследвания на изкуствения интелект — програмата за дама на Самюел (1959) се учи, като играе сама срещу себе си, коригирайки своята оценъчна функция въз основа на победи и загуби.

Тези нишки остават разделени до 80-те и 90-те години. Докторската работа на Сътън върху обучението по времеви разлики (temporal-difference learning, 1988) преодолява пропастта между теориите за учене при животните и динамичното програмиране на Белман. След това Уоткинс формализира Q-learning (1989), давайки на областта нейния първи изчистен алгоритъм за управление без модел (model-free control) с гаранции за сходимост. До момента, в който Сътън и Барто публикуват своя учебник през 1998 г. (актуализиран през 2018 г.), обучението с подкрепление (RL) се консолидира в разпознаваема дисциплина със собствена нотация, таксономия на задачите и набор от алгоритми.

Революцията в дълбокото обучение (deep learning) засегна RL през 2013-2015 г., когато алгоритъмът DQN на DeepMind се научи да играе игри на Atari от сурови пиксели. Тази демонстрация показа ясно, че RL алгоритмите, съчетани с невронни апроксиматори на функции, могат да се справят с многомерни реални проблеми. PPO последва през 2017 г. като практичен метод за градиент на политиката (policy gradient) и оттогава RL се разшири в роботиката, игровия ИИ, системите за препоръки и настройването на езикови модели (RLHF).

Важният извод е, че RL не е просто „машинно обучение с награди“. Това е отделна рамка за последователно вземане на решения в условия на несигурност, със собствени теоретични основи, извлечени от теорията на управлението, психоло-

гията и статистиката.

> **Прочетете повече:** Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition — Chapter 1.
> Свободен достъп: http://incompleteideas.net/book/the-book-2nd.html

---

1.2 Основни принципи — Агент, Действие, Среда (Agent, Action, Environment)

Цялата рамка на RL се основава на един прост цикъл. Един **агент** (agent) наблюдава текущото състояние (state) на дадена **среда** (environment), избира **действие** (action) и получава сигнал за **награда** (reward) плюс ново състояние. След това повтаря. Целта на агента е да научи **стратегия/политика** (policy) — съпоставяне от състояния към действия — която максимизира кумулативната награда във времето.

Няколко термина, които трябва да са ясни:

- **Състояние** (s): описание на средата в даден момент. При CartPole това е позицията на количката, скоростта на количката, ъгълът на стълба и ъгловата скорост на стълба. В покер би включвало раздадените карти и историята на залозите.
- **Действие** (a): това, което агентът може да направи. Дискретно (бутни наляво или надясно) или непрекъснато (приложи специфичен въртящ момент).
- **Награда** (r): скаларен сигнал за обратна връзка. Агентът я получава след всяко действие. Наградите могат да бъдат редки (sparse - само в края на играта) или гъсти (dense - на всяка стъпка).
- **Стратегия (Policy)** (π): стратегията на агента. Може да бъде детерминистична ($\pi(s) = a$) или стохастична ($\pi(a|s)$ дава разпределение на вероятностите над действията).
- **Функция на стойността (Value function)** ($V^\pi(s)$): очакваната кумулативна награда от състояние s нататък, ако агентът следва политика π. Това показва на агента колко добро е едно състояние в дългосрочен план, а не само непосредствено.
- **Фактор на дисконтиране (Discount factor)** (γ): число между 0 и 1, което контролира доколко агентът се интересува от бъдещите награди спрямо текущите. $\gamma = 0.99$ означава, че агентът е търпелив; $\gamma = 0.5$ означава, че е късоглед.

Критичното предизвикателство при дизайна в RL е **компромисът между из-**

следване и експлоатация (exploration-exploitation tradeoff). Агентът трябва да изследва непознати действия, за да открие потенциално по-добри стратегии, но също така трябва да експлоатира текущите си знания, за да събира награди. Твърде многото изследване губи време; твърде многото експлоатация води до засядане в локални оптимуми.

Съществуват две широки семейства от алгоритми. Методните, **базирани на стойността** (value-based, като DQN), научават стойността на състоянията или двойките състояние-действие и извеждат политика от тези стойности. Методните, **базирани на политиката** (policy-based, като PPO), научават политиката директно. И двете имат силни страни; голяма част от пейзажа на RL се свежда до тяхното ефективно комбиниране.

> **Прочетете повече:** OpenAI Spinning Up — "Key Concepts in RL"
> https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

---

1.3 Формулировката на MDP

Марковският процес на вземане на решения (Markov Decision Process, MDP) е формалната математическа рамка, върху която са изградени повечето RL алгоритми. Състои се от пет компонента:

- S: множество от състояния
- A: множество от действия
- $P(s'|s,a)$: функция на прехода — вероятността за преминаване в състояние s' при дадено състояние s и действие a
- $R(s,a,s')$: функция на наградата
- $\gamma \in [0,1)$: фактор на дисконтиране

Частта "Марков" означава, че бъдещето зависи само от текущото състояние, а не от историята на това как сте стигнали дотам. Това е силно предположение — и такова, което не е валидно в много интересни ситуации (като покера, където историята на залозите има значение). Ще се справим с това в Раздел 4.

Целта е да се намери оптимална политика π^*, която максимизира очакваната дисконтирана възвръщаемост (expected discounted return):

$$\pi^* = \arg\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

Ключовата рекурсивна връзка е **уравнението на Белман** (Bellman equation):

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) \left[R(s,a,s') + \gamma V^\pi(s')\right]$$

Това гласи: стойността на едно състояние е равна на очакваната незабавна награда плюс дисконтираната стойност на това, където ще се озовете. Цялото динамично програмиране, TD обучението и дълбокото RL се изграждат върху вариации на това уравнение.

> **Прочетете повече:** Sutton & Barto (2018). *Reinforcement Learning: An Introduction*, Chapter 3 — Finite Markov Decision Processes.
> Свободен достъп: http://incompleteideas.net/book/the-book-2nd.html

---

1.4 Информационни множества — Когато не виждате всичко

Стандартните MDP предполагат, че агентът може напълно да наблюдава състоянието. Реалните проблеми често нарушават това. В покера не виждате картите на опонента. В стратегическите игри в реално време има мъгла на войната (fog of war). Това са **частично наблюдаеми** (partially observable) среди.

Формалното разширение е **Частично наблюдаемият MDP** (POMDP), който добавя функция на наблюдение — агентът вижда наблюдения, които са шумни или непълни изгледи на истинското състояние. Но POMDP са известни с това, че са трудни за оптимално решаване.

В теорията на игрите същата идея е уловена чрез **информационните множества** (information sets). Едно информационно множество групира всички състояния на играта, които даден играч не може да различи едно от друго, въз основа на това, което е наблюдавал. В Kuhn Poker (3 карти, 2 играчи), когато държите Вале и не е имало залагания, вие сте в информационно множество — знаете своята карта, но не и тази на опонента, така че не можете да разберете дали истинското състояние е “Аз имам Вале, опонентът има Дама” или “Аз имам Вале, опонентът има Поп”.

Алгоритмите, които работят в тези ситуации — като CFR (Counterfactual Regret Minimization, разгледан в Стъпка 2) — оперират върху информационни множества, а не върху отделни състояния. Това е фундаментално различен изчислителен проблем от стандартния RL и затова изкуственият интелект за игра на игри изисква собствен набор от инструменти отвъд това, което DQN или PPO предоставят

готови за употреба.

Разбирането на информационните множества сега е важно, защото следващите стъпки (5–8) се занимават изцяло с игри с несъвършена информация (imperfect-information games). Алгоритмите се променят, но основната концепция остава: вие вземате решения въз основа на това, което можете да наблюдавате, разсъждавайки за това, което не можете.

> **Прочетете повече:** Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*, Chapter 5.
> Свободен достъп: http://www.masfoundations.org/download.html

---

1.5 Динамично програмиране — Когато имате модела

Динамичното програмиране (Dynamic programming - DP) е най-старият подход към решаването на MDP. То работи, когато имате пълно знание за средата: вероятностите за преход $P(s'|s,a)$ и функцията на наградата $R(s,a,s')$. На практика това означава, че познавате правилата на играта перфектно.

Двата основни DP алгоритъма са:

Итерация на политиката (Policy Iteration) редува две стъпки. Първо, *оценка на политиката* (policy evaluation): при дадена фиксирана политика, изчислява стойността на всяко състояние чрез многократно прилагане на уравнението на Белман до сходимост. Второ, *подобрение на политиката* (policy improvement): актуализира политиката, за да бъде алчна (greedy) по отношение на новоизчислените стойности. Повтаря се, докато политиката спре да се променя. Гарантирано е, че ще се сближи до оптималната политика.

Итерация на стойността (Value Iteration) е пряк път. Вместо напълно да оценява политиката, преди да я подобри, тя комбинира оценката и подобрението в едно обновяване. На всяка стъпка, за всяко състояние, избира действието, което максимизира очакваната възвръщаемост от една стъпка плюс дисконтираната стойност на следващото състояние. То се сближава до оптималната функция на стойността, от която се извлича оптималната политика.

DP е полезно, защото дава точни решения и има чисти гаранции за сходимост. Недостатъкът е очевиден: имате нужда от пълния модел и изчисленията се мащабират с броя на състоянията. За Kuhn Poker с 12 състояния DP е тривиално. За Go с 10^{170} състояния е невъзможно.

Причината, поради която DP има значение, въпреки че рядко го използваме директно, е, че всеки следващ метод — Монте Карло, TD обучение, DQN, PPO — може да се разбере като приближение към DP решението, когато моделът е неизвестен или пространството от състояния е твърде голямо. DP е теоретичният таван, до който практическите алгоритми се опитват да се доближат.

> **Прочетете повече:** Sutton & Barto (2018). *Reinforcement Learning: An Introduction*, Chapter 4 — Dynamic Programming.
> Свободен достъп: http://incompleteideas.net/book/the-book-2nd.html

---

1.6 Обучение по времеви разлики — Учене без модел

Обучението по времеви разлики (Temporal-difference learning - TD) е пробивът, който направи RL практично. За разлика от DP, то не се нуждае от модел на средата. За разлика от методите Монте Карло, то не трябва да чака до края на епизода, за да се учи. То актуализира оценките на стойността след всяка отделна стъпка, използвайки "буутстрапване" (bootstraping): текущата оценка на стойността на следващото състояние замества истинската бъдеща възвръщаемост.

Основното TD обновяване (TD(0)) за функцията на стойността е:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[r_t + \gamma V(s_{t+1}) - V(s_t) \right]$$

Терминът в скоби е **TD грешката** (TD error) — разликата между това, което се е случило (награда + оценена бъдеща стойност), и това, което е било предвидено. Ако TD грешката е положителна, състоянието е било по-добро от очакваното; ако е отрицателна, по-лошо.

Контролната версия на тази идея е **Q-learning** (Watkins, 1989):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Q-learning научава стойността на двойките състояние-действие и използва оператора max, за да бъде **извън политиката** (off-policy) — то научава за оптималната политика, дори докато следва проучвателна такава. Това е, което го прави толкова практично.

TD обучението е полезно в сравнение с DP, защото не се нуждае от модел. Полезно е в сравнение с Монте Карло, защото се учи онлайн (на всяка стъпка, не

на всеки епизод) и има по-ниска дисперсия (variance). Буутстрапингът въвежда известно отклонение (bias), но на практика този компромис е изключително благоприятен. Методът TD е основата, върху която са изградени DQN и повечето съвременни базирани на стойността алгоритми.

SARSA е братовчедът "вътре в политиката" (on-policy) на Q-learning — той се актуализира, като използва действието, което агентът всъщност е предприел впоследствие, вместо възможно най-доброто действие. Това го прави по-консервативен и понякога по-безопасен на практика, но не може да научи оптимална политика, докато следва неоптимална проучвателна политика.

> **Прочетете повече:** Sutton & Barto (2018). *Reinforcement Learning: An Introduction*, Chapter 6 — Temporal-Difference Learning.
> Свободен достъп: http://incompleteideas.net/book/the-book-2nd.html

---

1.7 DQN — Deep Q-Networks

DQN (Mnih et al., 2015) е алгоритъмът, който доказа, че дълбоките невронни мрежи могат да работят като апроксиматори на функции в RL в голям мащаб. Преди DQN, Q-learning беше ограничено до таблични настройки или ръчно създадени характеристики (features). DQN показа, че може да се научи да играе 49 игри на Atari от суров пикселен вход, надминавайки човешкото представяне в много от тях.

Как работи. DQN заменя Q-таблицата с невронна мрежа, която приема състояние като вход и извежда Q-стойности за всички действия. Мрежата се обучава чрез минимизиране на квадратичната TD грешка: разликата между предвидената Q-стойност и целта (награда + дисконтираната максимална Q-стойност на следващото състояние). На всяка стъпка агентът избира действието с най-високата Q-стойност (експлоатация) или случайно действие с вероятност ϵ (изследване).

Два трика правят това достатъчно стабилно, за да работи:

1. **Буфер за преиграване (Experience replay).** Вместо да се обучава върху последователни преходи (които са силно корелирани), преходите се съхраняват в голям буфер и се вземат на случаен принцип в мини-партиди (mini-batches). Това разрушава корелацията, намалява дисперсията и позволява всеки опит да се използва многократно. Това е същият принцип като разбъркването на тренировъчните ви данни при обучението с учител (supervised

learning).

2. **Целева мрежа (Target network).** Поддържа се отделно копие на Q-мрежата, което се актуализира само периодично (на всеки няколко хиляди стъпки или епизода). TD целта се изчислява с помощта на това замразено копие. Без него целта се измества при всяка стъпка на обучение — вие преследвате движеща се мишена, което причинява осцилации и разминаване (divergence). Целевата мрежа разделя актуализирането и стабилизира обучението.

Защо DQN има значение в сравнение с това, което е било преди: табличното Q-learning не може да се справи с многомерни състояния (изображения, непрекъснати наблюдения); известно е, че апроксимацията на функции само с Q-learning е нестабилна ("смъртоносната триада" от off-policy обучение + апроксимация на функции + буутстрапинг). Двата трика на DQN — буферът за преиграване и целевите мрежи — се справиха с нестабилността, правейки дълбокото базирано на стойността RL практично за първи път.

Ограничения. DQN се справя само с дискретни пространства от действия. То може да надцени Q-стойностите (което се адресира с Double DQN). То е off-policy, което е ефективно по отношение на извадките (sample-efficient), но може да доведе до стари данни в буфера за преиграване. И фундаментално, то научава алчна детерминистична политика — няма естествен начин за изразяване на стохастични политики, което има значение в игри, където смесването на стратегии е оптимално.

В нашата имплементация на Стъпка 1, DQN реши CartPole-v1 (постигайки средна стойност от 477.5 за 100 епизода срещу цел от 475) след ~1000 епизода на обучение. Най-влиятелните хиперпараметри бяха размерът на мрежата, минимумът на епсилон и честотата на синхронизация на целевата мрежа.

Бележка относно невронните мрежи в тази стъпка. Q-мрежата вътре в DQN е стандартен многослоен перцептрон (MLP), използван тук като практичен инструмент — функция, която съпоставя състояния с Q-стойности. За Стъпка 1 вътрешностите на невронната мрежа (инициализация на теглата, поток на градиента, активиращи функции) бяха третирани като черна кутия: използвахме стандартите по подразбиране на PyTorch и се фокусирахме върху самия RL алгоритъм. По-задълбочено разглеждане на архитектурите на невронните мрежи и това как те си взаимодействат с методите за намиране на равновесие ще дойде в Стъпка 5 (Апроксимация на равновесието с невронни мрежи).

Прочетете повече: Mnih, V. et al. (2015). "Human-level control through

deep reinforcement learning." *Nature*, 518(7540), 529–533.
arXiv: https://arxiv.org/abs/1509.06461

1.8 PPO — Proximal Policy Optimization

PPO (Schulman et al., 2017) възприема фундаментално различен подход към RL от DQN. Вместо да научава стойността на действията и да извлича политика от тези стойности, PPO **научава политиката директно** — невронна мрежа извежда разпределение на вероятностите над действията и мрежата се обучава да увеличава вероятността на действия, които водят до висока възвръщаемост.

Проблемът, който PPO решава. Методите за градиент на политиката са мощни на теория, но печално известни с това, че са нестабилни на практика. Основният алгоритъм REINFORCE страда от висока дисперсия и може да предприеме разрушително големи стъпки, които съсипват добрата политика. Trust Region Policy Optimization (TRPO, Schulman et al., 2015) адресира това, като ограничи колко може да се променя политиката при всяко обновяване, но TRPO изисква скъпа оптимизация от втори ред (изчисляване на Хесиана).

Как работи PPO. PPO запазва идеята за стабилност от TRPO, но заменя ограничението с много по-прост механизъм: **отрязване (clipping)**. Ключовото количество е съотношението на вероятностите $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$ — колко по-вероятно (или по-малкороятно) е новата политика да предприеме същото действие, което е предприела старата политика. PPO ограничава ("клипва") това съотношение до диапазона $[1-\epsilon, 1+\epsilon]$ (обикновено $\epsilon = 0.2$), което означава:

- Ако дадено действие се окаже добро (положително предимство - positive advantage) и съотношението нарасне отвъд $1+\epsilon$, градиентът се нулира — на политиката не е позволено да стане твърде нетърпелива за това действие.
- Ако действие се окаже лошо (отрицателно предимство) и съотношението падне под $1-\epsilon$, градиентът също се нулира — предотвратява се прекомерната корекция на политиката.

Това е **клипнатата сурогатна цел** (clipped surrogate objective) и е това, което кара PPO да работи. Получава стабилност, подобна на TRPO, използвайки само градиенти от първи ред (без Хесиан), което го прави значително по-евтин за изчисляване и по-лесен за имплементиране.

Архитектура. PPO е actor-critic (актьор-критик): **политическа мрежа (policy network)** (actor) извежда вероятностите за действие, а **мрежа за стойност-**

та (value network) (critic) оценява стойностите на състоянието. Мрежата за стойността позволява Обобщена оценка на предимството (Generalized Advantage Estimation - GAE), която осигурява оценки на предимството с ниска дисперсия за градиента на политиката. Двете мрежи се обучават едновременно.

Защо PPO има значение в сравнение с DQN: PPO естествено се справя както с дискретни, така и с непрекъснати пространства за действия. То може да научи стохастични политики, което е критично в игрови ситуации, където смесването на стратегии е необходимо. То е on-policy (използва пресни данни от текущата политика), което избягва проблеми със стари данни (stale data). И клипнатата цел го прави забележително стабилно — PPO е изборът по подразбиране за повечето RL приложения днес, включително RLHF за езикови модели.

Ограничения. Тъй като е on-policy, PPO е по-малко ефективно по отношение на извадките, отколкото off-policy методи като DQN (всеки опит се използва само за няколко епохи на обновяване, преди да бъде изхвърлен). Изисква се внимателна настройка на оценката на предимството (GAE lambda), бонуса за ентропия и архитектурата на мрежата. И все още може да се затруднява от редки награди (sparse rewards) — ако агентът рядко се сблъсква с успех, има малко сигнал, от който да се учи.

В нашата имплементация на Стъпка 1, PPO реши LunarLander-v3 (достигайки средна стойност от 202.2 за 100 епизода срещу цел от 200) при ~264K стъпки в средата. Решаващите фактори бяха капацитетът на мрежата (удвояване от [64,64] на [128,128]) и нормализирането на предимството.

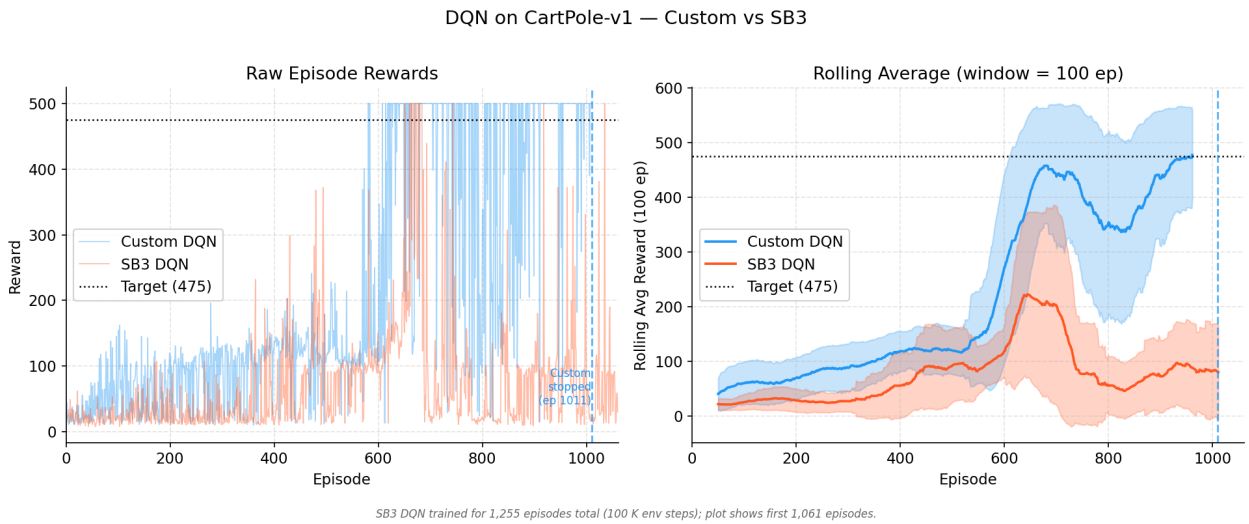
Бележка относно невронните мрежи в тази стъпка. Също както при DQN — политическата и мрежата за стойност тук са стандартни MLP, чиито вътрешности не бяха фокусът на тази стъпка. Ние възприехме практична позиция „то работи“ към невронните компоненти и се концентрирахме върху разбирането на логиката на алгоритъма на PPO (отрязване, GAE, структура на ролаут). По-дълбокото взаимодействие между дизайна на невронната мрежа и сходимостта на RL — особено в контекста на агенти за игра на игри — е отложено за Стъпка 5.

> **Прочетете повече:** Schulman, J. et al. (2017). “Proximal Policy Optimization Algorithms.”
> arXiv: https://arxiv.org/abs/1707.06347

1.9 Практическо валидиране — Собствени имплементации срещу Stable-Baselines3

За да проверим, че нашите имплементации от нулата действително работят, ги сравнихме със Stable-Baselines3 (SB3) — широко използвана, добре тествана RL библиотека. Както нашите имплементации, така и SB3, получиха съответстващи хиперпараметри и еквивалентни бюджети за обучение.

1.9.1 DQN на CartPole-v1



Фигура 1: DQN сравнение — Custom vs SB3

Нашият собствен DQN решава CartPole до епизод ~1011, достигайки пълзяща средна стойност за 100 епизода от 477.5 (над целта от 475). DQN от SB3, при дадени 750K стъпки в средата (17 176 епизода), така и не конвергира — неговата пълзяща средна остава около 30 и никога не се доближава до целта.

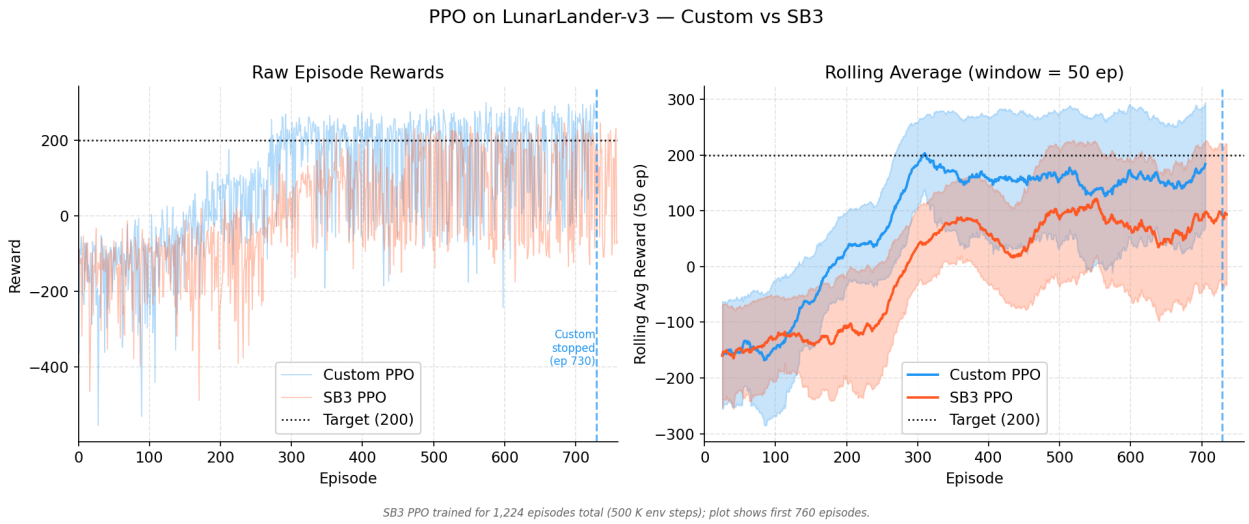
Разликата идва от три различия на ниво имплементация:

- **График за експлорация (Exploration schedule).** Нашият епсилон намалява на епизод (×0.995 всеки), концентрирайки изследването там, където има значение. SB3 намалява епсилон линейно на стъпки, което в среди с кратки епизоди като CartPole губи много епизоди в почти случаен режим.
- **Синхронизация на целевата мрежа.** Нашият код се синхронизира на всеки 5 епизода — адаптивен график, който дава по-бърза обратна връзка по време на ранното обучение. SB3 използва фиксиран интервал от 1000 стъпки.
- **Ранно спиране (Early stopping).** Нашият агент запазва най-добрия модел

и спира при пикова производителност, избягвайки катастрофално забравяне (catastrophic forgetting). SB3 изпълнява пълния бюджет и политиката осцилира.

Това са същински алгоритмични избори, а не нечестност в хиперпараметрите. Стойностите по подразбиране на SB3 са проектирани за стабилна производителност в разнообразни области с дълго обучение, като игрите на Atari — не за кратки епизоди в среда на класическо управление (classic control) като CartPole; нашите специфични за задачата избори просто работят по-добре на CartPole.

1.9.2 PPO на LunarLander-v3



Фигура 2: PPO сравнение — Custom vs SB3

Нашият собствен PPO преминава целта от 200 около епизод 543 (264K стъпки). PPO от SB3 със същия бюджет от 500K стъпки достига връх при най-добра пълзяща средна от 100 епизода от 131.2 — все още се покачва, но не е конвергирал.

Разликата тук е по-малка, отколкото при DQN. И двете използват идентични PPO хиперпараметри. Оставащата разлика вероятно идва от ортогоналната инициализация на теглата на SB3 и вграденото нормализиране на наблюденията, които помагат при дълги серии (runs), но могат да забавят ранната сходимост. Ако му се даде повече тренировъчен бюджет (1–2 милиона стъпки), SB3 PPO вероятно би достигнал целта.

1.9.3 Изводи

Една изчистена имплементация от нулата е 300–500 реда в сравнение с ~10K на SB3. Компромисът: по-малко функции, но пълна прозрачност и възможност за дебъгване. За изследователски цели — особено работата по експлоатация на опонента (opponent exploitation), планирана за Стъпки 7–8 — персонализираните имплементации позволяват хирургични модификации (напр. инжектиране на наблюдение на „състояние на убеждение“ - belief-state), които биха изисквали дълбоко създаване на подкласове в SB3. Тази практическа гъвкавост е причината да изграждаме от нулата.